



Реверсь своих, чтоб *чужим неповадно было* шифруем прошивку на SOC RK35* без NDA

Никита Фирсов

Expert, Positive Labs

tg: @Ahteesh0



Who am I

Positive Labs Expert

Reverse engineer

Hardware security researcher

Homelabber



Никита Фирсов

tg: @Ahteesh0



Agenda

- Introduction
 - SecureBoot
 - EncryptedBoot
 - Other chips
 - Bug mitigation
 - How you can use it



Intro

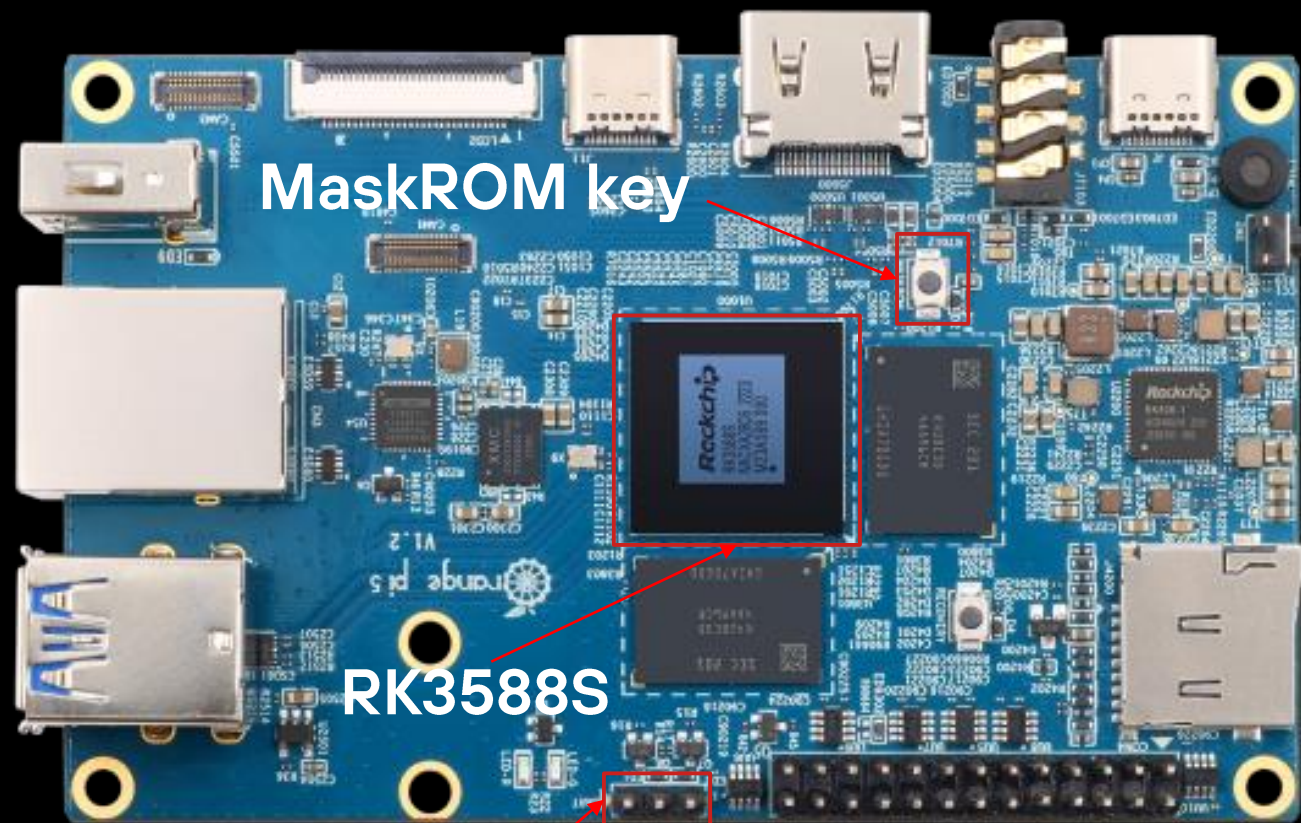
что, где, зачем

Технологии защиты

Secure & Encrypted boot

Первоначальное исследование:
RK3588S

Портировано на:
RK3566
RK3576



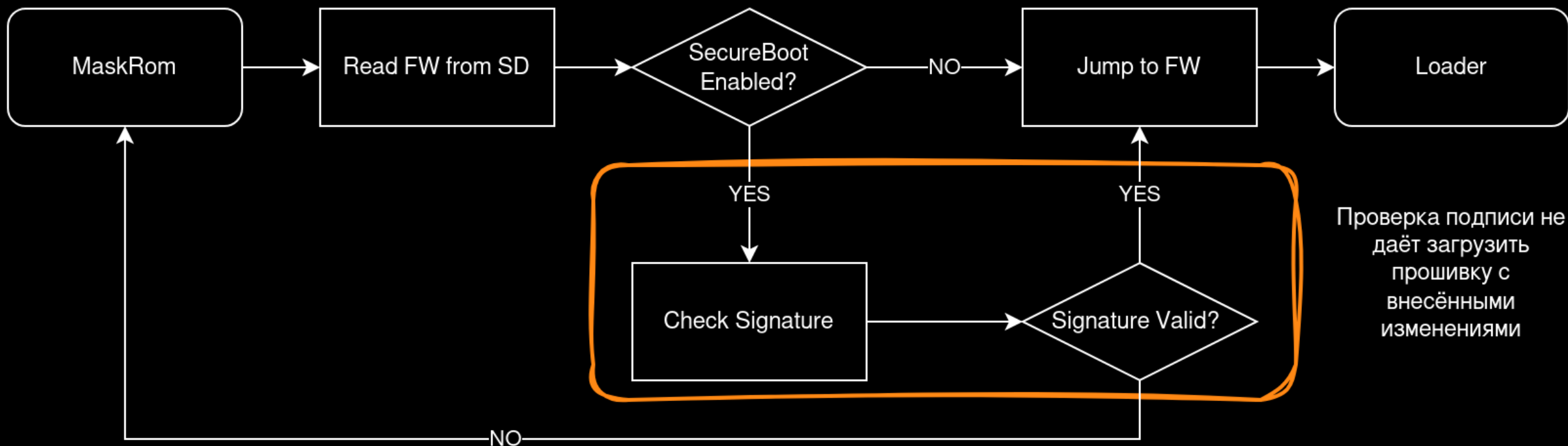
MaskROM key

RK3588S

UART

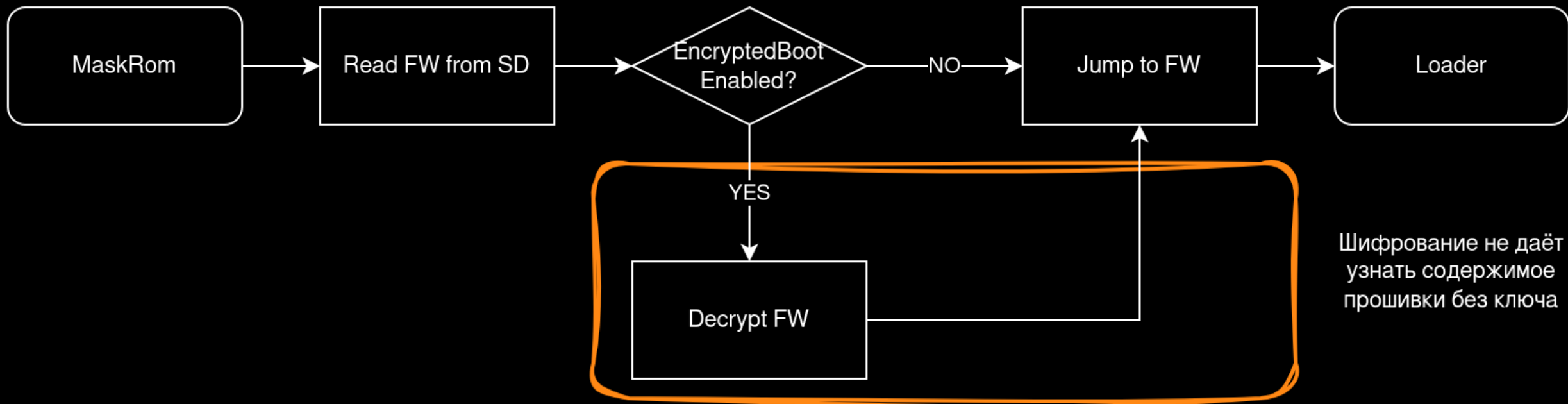
SecureBoot

ОСНОВА ОСНОВ



EncryptedBoot

Хочешь спрятать - зашифруй

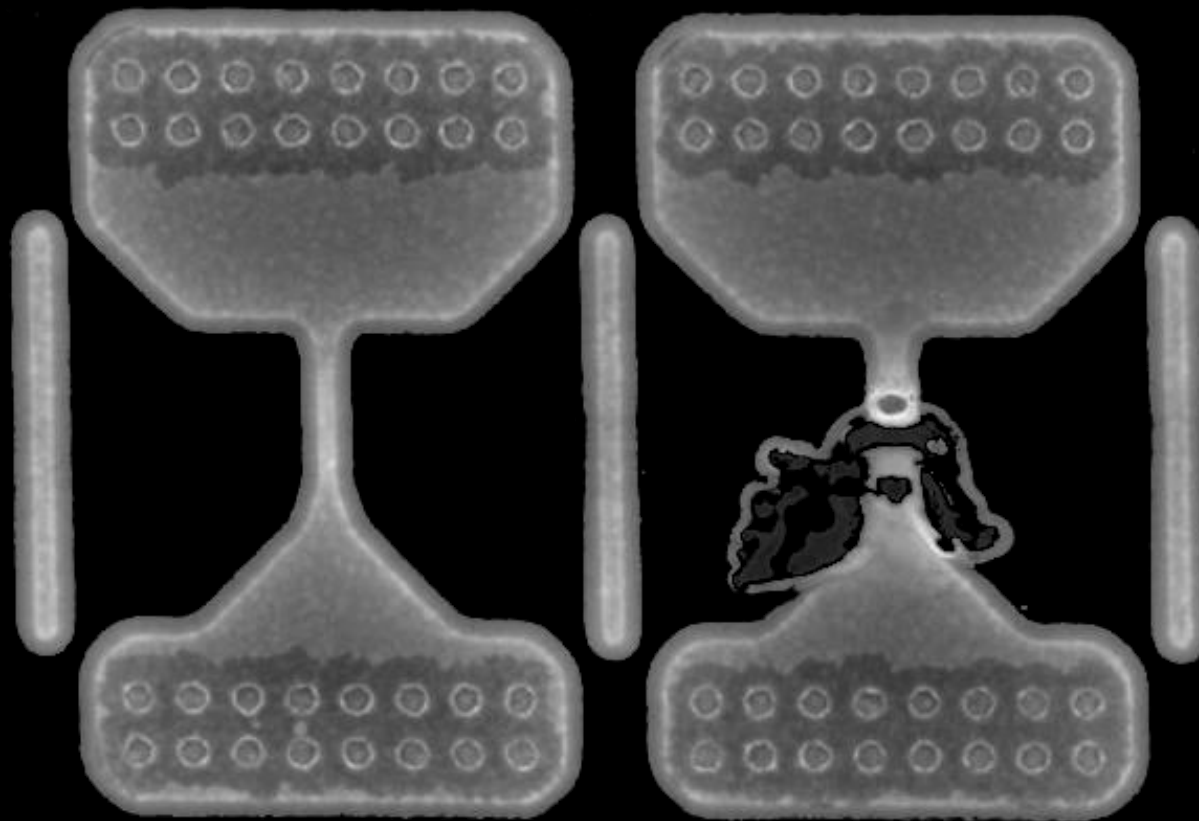


ОТР

Они же фьюзы

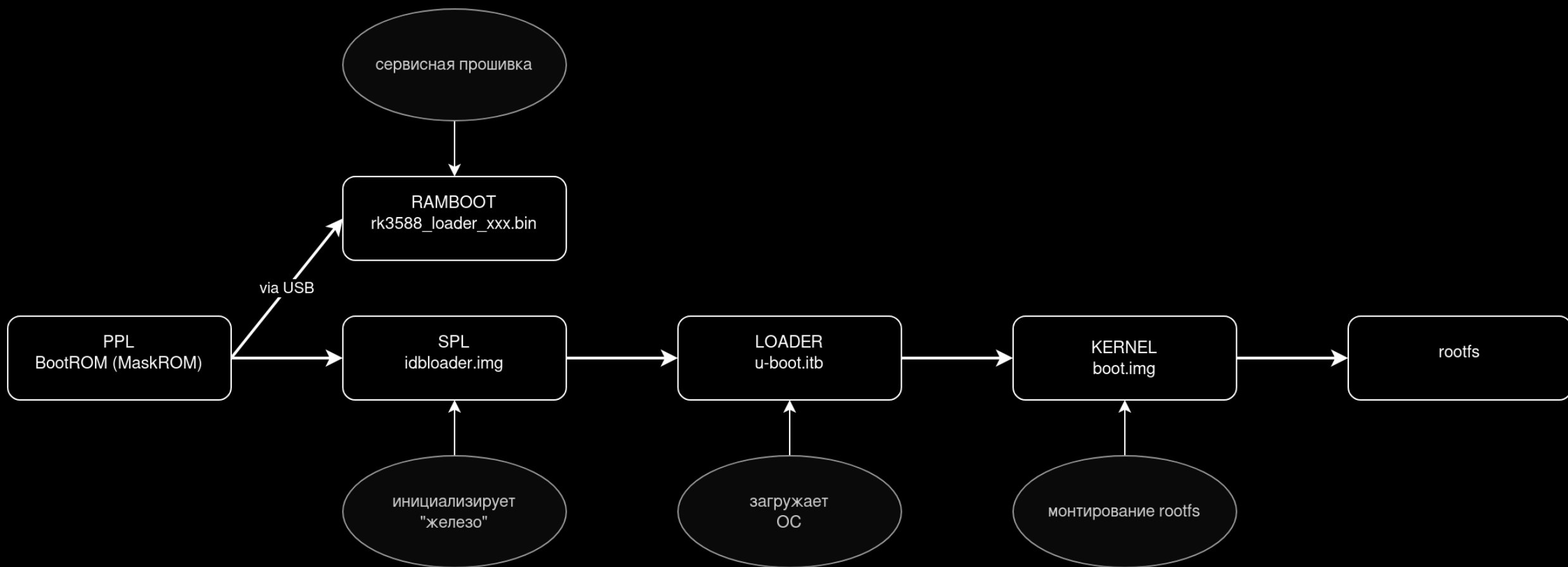
One-Time Programmable

Можно записать только 1 раз
Обычно это правило применимо к
отдельным битам, можно поменять
любой 0 на 1, но нельзя поменять 1
на 0



Boot sequence

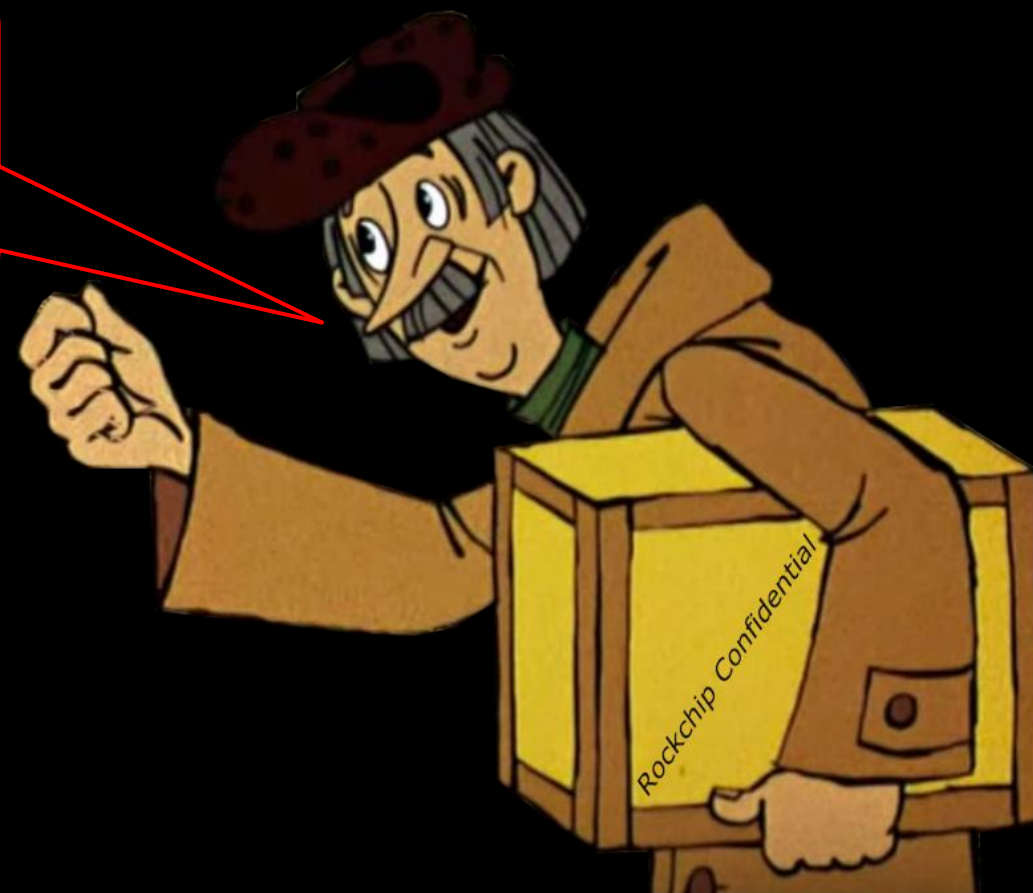
Загрузка шаг за шагом



NDA hell

технология у нас есть...

*Принёс eBoot для вашего RK3588.
Только я вам его не отдам,
вы NDA не подписывали*





SecureBoot

не пущать чужое

Tools & Docs

не всё так плохо

Tools:

- rkbin
- rkdeveloptool

Docs:

- Datasheet
- Technical Reference Manual



rockchip-linux / rkbin Public

<> Code Pull requests 1 Actions Projects Security Insights

master 1 Branch 0 Tags

owenchenw

- RKBOOT
- RKTRUST
- bin
- doc/release
- img/rk1x
- scripts
- tools
- .gitignore
- LICENSE
- README
- RKBOOT.ini

rockchip-linux / rkdeveloptool Public

<> Code Issues 39 Pull requests 16 Actions

master 1 Branch 0 Tags

liuyi v1.32: 📄

- cfg update the build sys
- .gitignore update the build sys
- 99-rk-rockusb.rules rk3128: add rk3128
- CMakeLists.txt Adapt to CMake.
- DefineHeader.h v1.32:
- Endian.h rkDevelopTool: initi
- Makefile.am use rm instead of \$
- Property.hpp rkDevelopTool: initi
- RKBoot.cpp v1.32:

Rockchip RK3588 Technical Reference Manual

Revision History

Date	Revision	Description
2022-03-09	1.0	Initial Release

Copyright 2022 © Rockchip Electronics Co., Ltd. 1

MaskRom

Как его достать

DumpROM.s

```
BASE=0x30076d8
hexdump=0x300A644
usleap=0x300439C
```

start:

```
MOV    X19, 0xffff0000
MOV    X20, 0xffff8000
```

step:

```
ADR    X0, .fmt
MOV    X1, X19
MOV    W2, 0x1
MOV    W3, 0x100
```

```
BL     (hexdump+start-BASE)
```

```
MOV    W0, 0x10000
BL     (usleap+start-BASE)
ADD    X19, X19, 0x100
CMP    X19, X20
BNE    step
```

loop:

```
B loop
```

```
.fmt:
.ascii "dump: \x00"
```

The following table show the boot address when before remap and after remap

Table 1-2 Address Remapping

remap[1:0]=2'b00		remap[1:0]=2'b01		remap[1:0]=2'b10	
		not accessible	BootRom(32KB)	not accessible	BootRom(32KB)
0xFFFF0000	BootRom(32KB)	0xFFFF0000	PMU_SRAM(8KB)	0xFFFF0000	SYSTEM_SRAM(256KB)
0xFF100000	PMU_SRAM(8KB)	0xFF100000	PMU_SRAM(8KB)	0xFF100000	PMU_SRAM(8KB)
0xFF000000	SYSTEM_SRAM(1MB)	0xFF000000	SYSTEM_SRAM(1MB)	0xFF000000	SYSTEM_SRAM(1MB)

Boot1 Release Time: Feb 24 2022 10:23:56, version: 1.05 USB B00T

ChipType = 0x32, 469

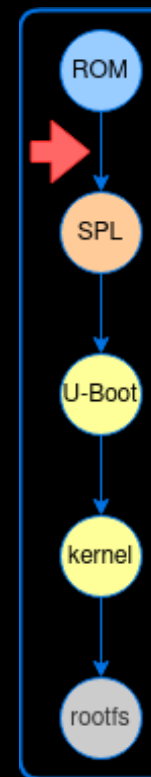
SecureMode = 0

atags_set_bootdev: ret:(0)

UsbBoot ...1050

powerOn 3334

```
dump: 00000000ffff0000 + 0x0:0xa0,0x00,0x38,0xd5,0x00,0x3c,0x40,0x92,0x1f,0x00,0x00,0xf1,0xa0,0x01,0x00,0x54,
dump: 00000000ffff0000 + 0x10:0x80,0x06,0x00,0x58,0x21,0x00,0x80,0x52,0x01,0x00,0x00,0xb9,0x5f,0x20,0x03,0xd5,
dump: 00000000ffff0000 + 0x20:0x82,0x06,0x00,0x18,0xe0,0x05,0x00,0x58,0x01,0x00,0x40,0xb9,0x3f,0x00,0x02,0x6b,
dump: 00000000ffff0000 + 0x30:0x61,0xff,0xff,0x54,0xa1,0x05,0x00,0x58,0x20,0x00,0x40,0xb9,0x00,0x00,0x1f,0xd6,
dump: 00000000ffff0000 + 0x40:0xe0,0x03,0x1f,0xaa,0xe1,0x03,0x1f,0xaa,0xe2,0x03,0x1f,0xaa,0xe3,0x03,0x1f,0xaa,
dump: 00000000ffff0000 + 0x50:0xe4,0x03,0x1f,0xaa,0xe5,0x03,0x1f,0xaa,0xe6,0x03,0x1f,0xaa,0xe7,0x03,0x1f,0xaa,
dump: 00000000ffff0000 + 0x60:0xe8,0x03,0x1f,0xaa,0xe9,0x03,0x1f,0xaa,0xea,0x03,0x1f,0xaa,0xeb,0x03,0x1f,0xaa,
dump: 00000000ffff0000 + 0x70:0xec,0x03,0x1f,0xaa,0xed,0x03,0x1f,0xaa,0xee,0x03,0x1f,0xaa,0xef,0x03,0x1f,0xaa,
dump: 00000000ffff0000 + 0x80:0xf0,0x03,0x1f,0xaa,0xf1,0x03,0x1f,0xaa,0xf2,0x03,0x1f,0xaa,0xf3,0x03,0x1f,0xaa,
dump: 00000000ffff0000 + 0x90:0xf4,0x03,0x1f,0xaa,0xf5,0x03,0x1f,0xaa,0xf6,0x03,0x1f,0xaa,0xf7,0x03,0x1f,0xaa,
dump: 00000000ffff0000 + 0xa0:0xf8,0x03,0x1f,0xaa,0xf9,0x03,0x1f,0xaa,0xfa,0x03,0x1f,0xaa,0xfb,0x03,0x1f,0xaa,
dump: 00000000ffff0000 + 0xb0:0xfc,0x03,0x1f,0xaa,0xfd,0x03,0x1f,0xaa,0xfe,0x03,0x1f,0xaa,0xa0,0x00,0x00,0x58,
dump: 00000000ffff0000 + 0xc0:0x1f,0x00,0x00,0x91,0xa0,0x00,0x00,0x58,0x00,0xc0,0x1e,0xd5,0x09,0x03,0x00,0x14,
dump: 00000000ffff0000 + 0xd0:0x00,0x10,0x00,0xff,0x00,0x00,0x00,0x00,0x48,0xff,0xff,0x00,0x00,0x00,0x00,
dump: 00000000ffff0000 + 0xe0:0x04,0x00,0x00,0xff,0x00,0x00,0x00,0x00,0x08,0x00,0x00,0xff,0x00,0x00,0x00,
dump: 00000000ffff0000 + 0xf0:0xaf,0xbe,0xad,0xde,0x00,0x00,0x00,0x00,0x03,0x00,0x01,0x2a,0x63,0x04,0x00,0x12,
```



MaskRom

их 2



Чтоб увидеть SecureBoot нужен секретный MaskROM?

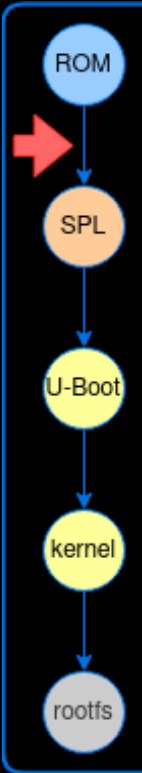
Чтоб увидеть секретный MaskROM нужен SecureBoot?

1.2 System Boot

RK3588 provides system boot from off-chip devices such as SDMMC card, eMMC memory, serial Nand or Nor flash. When boot code is not ready in these devices, also provide system code download into them by USB OTG interface. All of the boot code will be stored in internal BootRom. The following is the whole boot procedure for boot code, which will be stored in BootRom in advance.

The following features are supported.

- Support system boot from the following device:
 - Serial Nor Flash, 1bit or 4bits data width(device layout in FSPI IO)
 - Serial Nand Flash, 1bit data width(device layout in FSPI IO)
 - eMMC Interface, 8bits data width
 - SDMMC Card, 4bits data width
 - RK3588 has two bootwom:
 - Normal Bootrom in non-secure world, which CPU_S and CPU_NS both can access
 - Secure Bootrom in secure world, only CPU_S can access
- Support system code download by USB OTG



RamBoot

SecureInit и SecureWriteOTP



SecureInit

```
printf("Head flags=0x%x\n", (a1[0xC] | (a1[0xD] << 8)) | ((a1[0xE] | (a1[0xF] << 8)) << 0x10));
if ( !SecureCheckHeader(a1) )
{
    if ( (a1[0xD] & 0x20) == 0 )
    {
        dword_3332DD0[0] = 1;
        unk_3122140 = 1;
    }
    LABEL_13:
    sub_3001250(0x3000800, a1, 0x368u);
    goto LABEL_14;
}
if ( SecureWriteOTP(a1) )
{
    ret = 0xFFFFFFFFFA;
    goto LABEL_15;
}
dword_3332DD0[0] = 1;
LABEL_10:
if ( sub_3006AF0(a1) )
{
    ret = 0xFFFFFFFFFB;
    goto LABEL_15;
}
unk_3122140 = 1;
goto LABEL_13;
}
ret = 0xFFFFFFFFFC;
LABEL_15:
printf("SecureInit ret = %d, SecureMode = %d\n", ret, dword_3332DD0[0]);
```

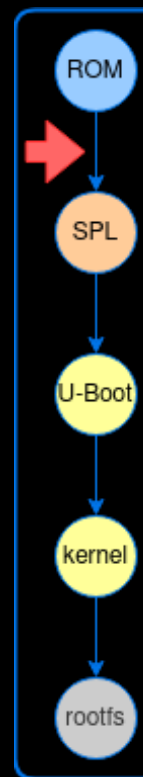
Проверка корректности
подписи

SecureWriteOTP

```
printf("SecureWriteOTP\n");
err = RkOTPRead(v5, 0x270u, 8u);
if ( (err & 0x80000000) != 0 )
    goto ERROR;
if ( *v5 )
    sub_300A644("otp no empty!", v5, 4u, 8u);
err = sha256(data + 0x200, 0x230u, key_hash);
if ( err )
    goto ERROR;
err = RkOTPWrite(key_hash, 0x270u, 8u);
if ( err )
    goto ERROR;
err = RkOTPRead(v5, 0x270u, 8u);
if ( err )
    goto ERROR;
if ( sub_3001178(v5, key_hash, 0x20u) )
{
    sub_300A644("key_hash:", key_hash, 4u, 8u);
    sub_300A644("otp_data:", v5, 4u, 8u);
    err = 0xFFFFFFFFFE;
}
ERROR:
printf("otp write error: %d!!!\n", err);
return err;
}
if ( (*(data + 0xC) & 0xF0) == 0x20 )
    v3 = 0x30FF;
else
    v3 = 0xFF;
a1 = v3;
if ( some_flag )
{
    v3 |= 0x300u;
    a1 = v3;
}
printf("enable secure flag:0x%x\n", v3);
err = RkOTPWrite(&a1, 8u, 1u);
if ( err )
    goto ERROR;
v7 = 0;
err = RkOTPRead(&v7, 8u, 1u);
if ( err )
    goto ERROR;
if ( v7 != a1 )
{
    printf("otp flag:%x, error:\n", v7);
    err = 0xFFFFFFFF;
    goto ERROR;
}
printf("otp write key success!!!\n");
```

Запись хэша ключа

Запись флагов,
включающих
SecureBoot



Write OTP

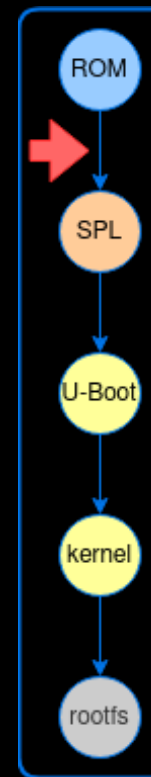
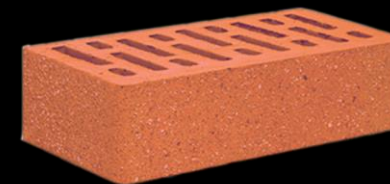
Как я потерял первую апельсинку

SecureInit.s

```
FILE=0x6ed8
BASE=0x30076d8
SecureWriteOTP=0x3006be4
SecureCheckHeader=0x3006934
printf=0x300A1F4
start:
LDR      X0,=0x3008bd0
BL       (SecureCheckHeader+start-BASE)
LDR      X0,=0x3008bd0
BL       (SecureWriteOTP+start-BASE)
LDR      X0,=0x3008bd0
BL       (printf+start-BASE)
loop:
B loop
```

```
out
Boot1 Release Time: Feb 24 2022 10:23:56, version: 1.05 USB B00T
ChipType = 0x32, 466
SecureMode = 1
atags_set_bootdev: ret:(0)
UsbBoot ...1049
powerOn 3363
SecureCheckHeader -1
RKSS
```

Игнорирование ошибок
к добру не приводит.
Особенно с фьюзами.



Write OTP

some_flag



```
__int64 __fastcall SecureCheckHeader(unsigned __int8 *data)
{
    __int64 v1; // x1
    unsigned __int8 *key; // x2
    unsigned int ret_code; // w20

    if ( ((*data | (data[1] << 8)) | ((data[2] | (data[3] << 8)) << 0x10)) != 'SSKR' ) // RKSS
        goto RET_m1;
    key = data + 0x200;
    if ( !some_flag )
    {
        LABEL_6:
            ret_code = sub_3006B90(data, v1, key, (data + 0x600));
            goto RET;
    }
    if ( some_buff[0] != 0x4B504B52 ) // RKPK
    {
        RET_m1:
            ret_code = -1u;
            goto RET;
    }
    ret_code = sub_3006B90(some_buff, v1, key, &some_buff[0x180]);
    if ( !ret_code )
    {
        key = &some_buff[0x10];
        goto LABEL_6;
    }
}
RET:
    printf("SecureCheckHeader %d\n", ret_code);
    return ret_code;
}
```

Опускаем some_flag

// RKPK Проверям код возврата ;)

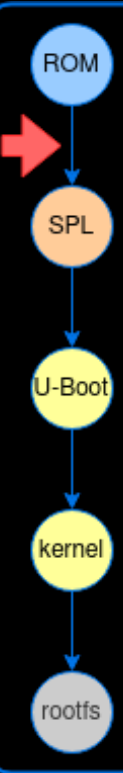
UsbBoot ...1050

powerOn 3301

SecureCheckHeader 0
Succes!

SecureInit_Fixed.s

```
FILE=0x6ed8
BASE=0x30076d8
SecureWriteOTP=0x3006be4
SecureCheckHeader=0x3006934
printf=0x300A1F4
start:
    LDR        X0,=0x3120030
    MOV        W8, 0
    STR        W8, [X0,4]
    MOV        W8, 0x100
    STR        W8, [X0]
    LDR        X0,=0x3008bd0
    BL         (SecureCheckHeader+start-BASE)
    CBNZ       W0, fail
    LDR        X0,=0x3008bd0
    BL         (SecureWriteOTP+start-BASE)
    CBNZ       W0, fail
    ADR        X0,.ok_msg
    BL         (printf+start-BASE)
    B loop
fail:
    ADR        X0,.fail_msg
    BL         (printf+start-BASE)
    B loop
    B loop
.ok_msg: .ascii "Succes!\n\x00"
.fail_msg: .ascii "Fail :(\n\x00"
```



Signed SPL

rk_sign_tool - работает



DDR Version V1.08 20220617

LPDDR4X, 2112MHz

channel[0] BW=16 Col=10 Bk=8 CS0 Row=16 CS=1 Die BW=16 Size=1024MB

channel[1] BW=16 Col=10 Bk=8 CS0 Row=16 CS=1 Die BW=16 Size=1024MB

channel[2] BW=16 Col=10 Bk=8 CS0 Row=16 CS=1 Die BW=16 Size=1024MB

channel[3] BW=16 Col=10 Bk=8 CS0 Row=16 CS=1 Die BW=16 Size=1024MB

Manufacturer ID:0x1 Samsung

CH0 RX Vref:34.7%, TX Vref:18.8%,0.0%

CH1 RX Vref:34.7%, TX Vref:18.8%,0.0%

CH2 RX Vref:30.7%, TX Vref:21.8%,0.0%

CH3 RX Vref:31.7%, TX Vref:20.8%,0.0%

change to F1: 528MHz

change to F2: 1068MHz

change to F3: 1560MHz

change to F0: 2112MHz

out

U-Boot SPL board init

U-Boot SPL 2017.09-orangepi (Jun 01 2023 - 21:22:25)

unknown raw ID 0 0 0

unrecognized JEDEC id bytes: 00, 00, 00

Trying to boot from MMC1

Trying fit image at 0x4000 sector

Verified-boot: 1

Verified-boot requires CONFIG_SPL_FIT_SIGNATURE enabled

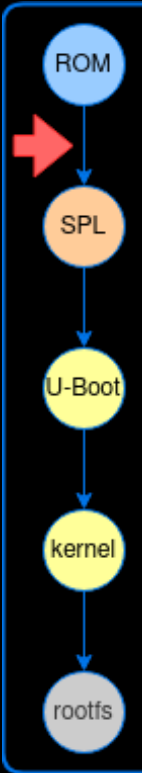
ERROR ### Please RESET the board

Reset the board to bootrom

```
/* if board sigs verify required, check self */
if (fit_board_verify_required_sigs() &&
    !IS_ENABLED(CONFIG_SPL_FIT_SIGNATURE)) {
    printf("Verified-boot requires CONFIG_SPL_FIT_SIGNATURE enabled\n");
    hang();
}
```

Эту строчку выводит SPL а не MaskROM,
значит SPL подписан правильно

SPL хочет проверить
подпись, но не умеет.



Signed Kernel FIT

Как его собрать

Flattened Image Tree — формат загрузочного образа похожий на flattened device tree

Параметры образа:
содержимое,
подпись

Параметры бинарей: путь к файлу, адрес загрузки, хэш

Конфиг для сборки

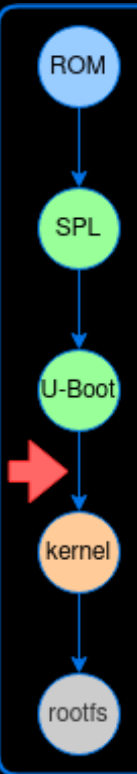
```
configurations {
    default = "conf";

    conf {
        rollback-index = <0x00>;
        fdt = "fdt";
        kernel = "kernel";
        ramdisk = "ramdisk";

        signature {
            algo = "sha256,rsa2048";
            padding = "pss";
            key-name-hint = "dev";
            sign-images = "fdt", "kernel", "ramdisk";
        };
    };
};

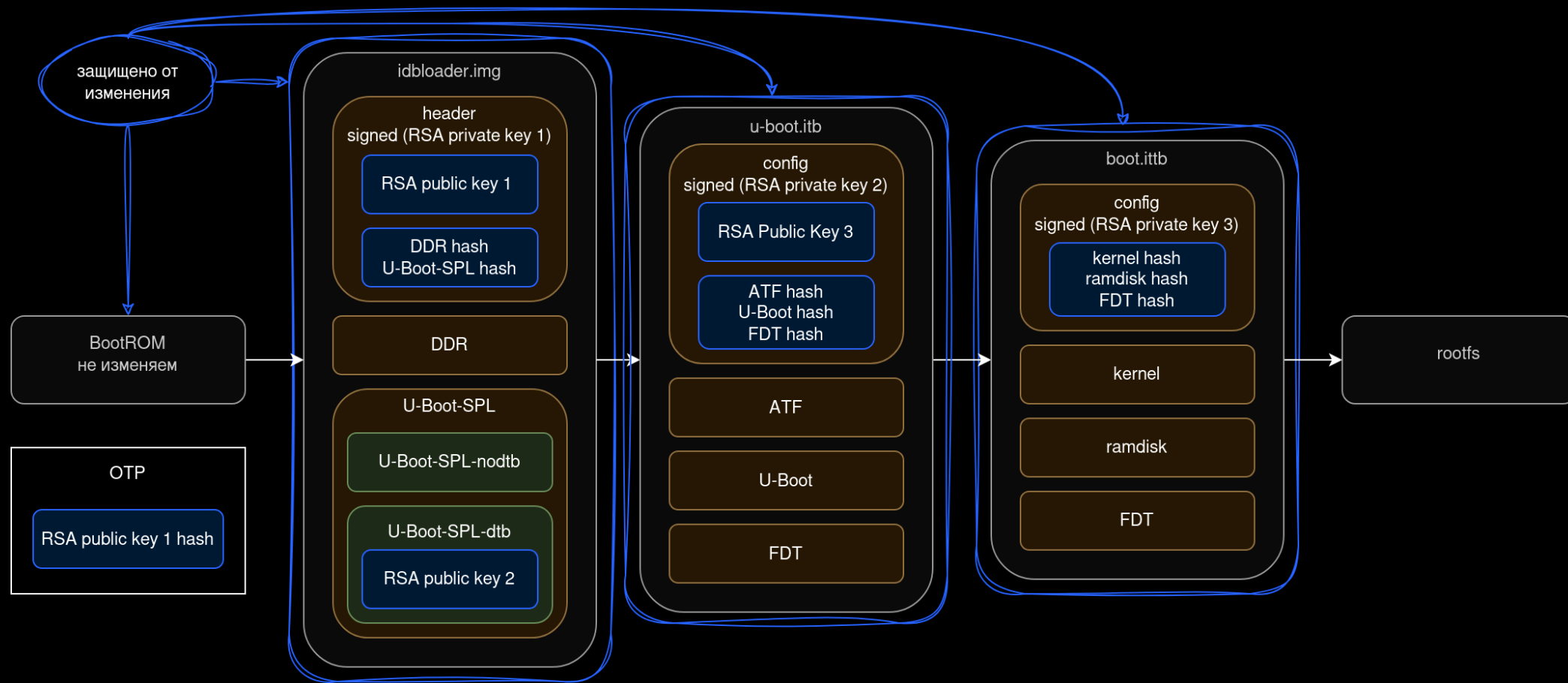
images {
    kernel {
        data = /incbin("./Image");
        type = "kernel";
        arch = "arm64";
        os = "linux";
        compression = "NONE";
        load = <0x00400000>;
        entry = <0x00400000>;

        hash {
            algo = "sha256";
        };
    };
    fdt {
        data = /incbin("./rk3588s-orangepi-5.dtb");
        type = "flat_dt";
    };
};
```



SecureBoot

Как выглядит цепочка





EncryptedBoot

спрятать своё

U-boot XOR

Начнём с простого



arch/arm/mach-rockchip/fit_misc.c — post process

```
void board_fit_image_post_process(void *fit, int node, ulong *load_addr,
                                  ulong **src_addr, size_t *src_len, void *spec)
{
    fit_decrypt_image(fit, node, load_addr, src_addr, src_len, spec);
    #if CONFIG_IS_ENABLED(MISC_DECOMPRESS) || CONFIG_IS_ENABLED(GZIP)
    fit_decomp_image(fit, node, load_addr, src_addr, src_len, spec);
    #endif
    #if CONFIG_IS_ENABLED(USING_KERNEL_DTB)
    /* Avoid overriding processed(overlay, hw-dtb, ...) kernel dtb */
    if (fit_image_check_type(fit, node, IH_TYPE_FLATDT)) {
        if ((gd->flags & GD_FLG_KDTB_READY) && !gd->fdt_blob_kern) {
            *src_addr = (void *)gd->fdt_blob;
            *src_len = (size_t)fdt_totalsize(gd->fdt_blob);
        } else {
            printf("    Using fdt from load-in fdt\n");
        }
    }
    #endif
}
```

decompress — хорошая основа для decrypt

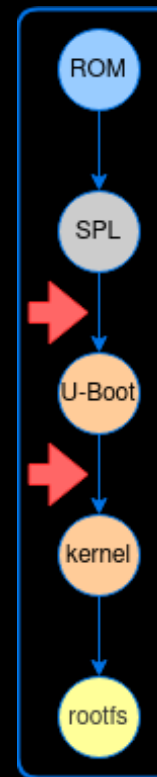
Поддержка разных методов шифрования
(как в desomp — разных алгоритмов сжатия)
пригодится когда будем добавлять AES

arch/arm/mach-rockchip/fit_misc.c — decrypt

```
#define FIT_ENC_KEY_SIZE 16
int fit_decrypt_image(void *fit, int node, ulong *load_addr,
                     ulong **src_addr, size_t *src_len, void *spec)
{
    uint8_t img_enc;
    u64 len = *src_len;
    fit_image_get_enc(fit, node, &img_enc);
    printf("Encryption type=%d\n", img_enc);
    printf("src_addr=%p, src_addr=%lx, len=%lld\n", *src_addr, *src_addr, len);
    if (img_enc == IH_ENC_NONE)
        return 0;
    if (img_enc != IH_ENC_XOR) {
        printf("Image decryption failed: unknown encryption\n");
        return -1;
    }

    uint8_t key[FIT_ENC_KEY_SIZE] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0xA, 0xB, 0xC, 0xD, 0xE, 0xF };
    uint8_t *xor_src = (uint8_t *)(*src_addr);
    for (u64 i = 0; i < len; ++i) {
        xor_src[i] = xor_src[i] ^ key[i % FIT_ENC_KEY_SIZE];
    }
    *src_addr = (ulong *)*load_addr;
    *src_len = len;
    return 0;
}
```

XOR в 5 строчек. Ну где здесь можно ошибиться?



U-boot XOR OTP

Спрячем ключ



Да-да, это сорцы, верим

drivers/misc/rk3588-secure-otp.S

```
.LVL27:
#APP
// 68 "drivers/misc/rk3588-secure-otp.c" 1
    dmb sy
    // 0 "" 2
    #NO_APP
    ldr    x1, [x20]

.LVL28:

.LVL50:
    ccmp   w0, 7, 0, ne
    bls    .L26
    .loc 1 102 0 is_stmt 0 discriminator 1
    sub    w0, w20, #2496
    cmp    w0, 63
    bls    .L26
    .loc 1 102 0 discriminator 2
    sub    w0, w20, #416
    mov    x19, 0
    cmp    w0, 95
    bls    .L23
    bl     .L23
    .loc 1 106 0 is_stmt 1
    mov    w21, -1
    .loc 1 105 0
    mov    w1, w20
    adrp   x0, .LC3
    add    x0, x0, :lo12:.LC3
    bl     printf
```

Ограничения для слабаков

Коварный патч

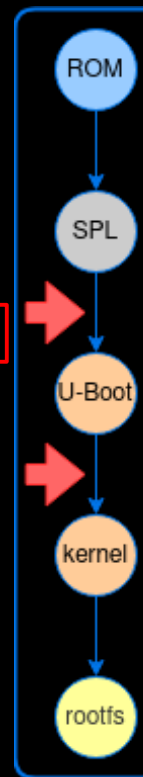
IDA: rk3588-secure-otp.o

```
CCMP      W0, #7, #0, NE
B.LS      loc_2B0
SUB       W0, W20, #0x9C0
CMP       W0, #0x3F ; '?'
B.LS      loc_2B0
SUB       W0, W20, #0x1A0
MOV       X19, #0
CMP       W0, #0x5F ; '_'
B.LS      loc_2B4
MOV       W21, #0xFFFFFFFF
MOV       W1, W20
ADRL      X0, aPleaseInputCor ; "Please input correct addr, offset is %x"...
BL        printf
```

IDA лучше "сорцов"

```
platdata = dev_get_platdata(dev);
v8 = offset != 0x20 && (offset - 0x150) > 7;
if ( v8 && (offset - 0x9C0) > 0x3F )
{
    v9 = 0;
    if ( (offset - 0x1A0) <= 0x5F )
        goto LABEL_13;
    byte = 0xFFFFFFFF;
    printf("Please input correct addr, offset is %x\n", offset);
}
else
{
    v9 = 0;
LABEL_13:
    while ( size > v9 )
    {
        v12 = offset + v9;
        v13 = buf + v9++;
        byte = rk3588_secure_otp_read_byte(platdata, v13, v12);
        if ( byte )
```

Особенно если
жмакнуть F5



U-boot XOR FIT

ну не так нет

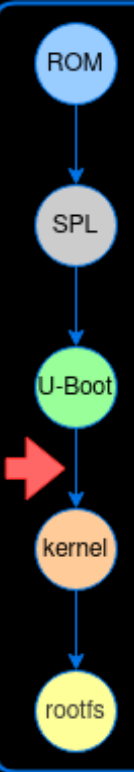


arch/arm/mach-rockchip/fit_misc.c : fit_decrypt_image

```
if(img_enc==IH_ENC_OTP){
    struct udevice* dev = misc otp get device(OTP S);
    misc_otp_read(dev, KEY_OTP_ADDR, key, FIT_ENC_KEY_SIZE);
}
if(img_enc==IH_ENC_FDT){
    const void *blob = gd_fdt_blob();
    int key_node=fdt subnode offset(blob, 0, "encryption");
    key =(uint8_t *) fdt_getprop(blob, key_node, "key", NULL);
}
```

Читаем ключ из OTP

Или берём из FDT



U-boot AES

XOR удобен для тестов но небезопасен



```
int rk_crypto_ae(struct udevice *dev, u32 algo, u32 mode,
                 const u8 *key, u32 key_len, const u8 *nonce, u32 nonce_len,
                 const u8 *in, u32 len, const u8 *aad, u32 aad_len,
                 u8 *out, u8 *tag)
{
    u32 rk_mode = RK_GET_RK_MODE(mode);
    int ret;

    if (!IS_AE_MODE(rk_mode))
        return -EINVAL;

    if (algo != CRYPTO_AES && algo != CRYPTO_SM4)
        return -EINVAL;

    /* RV1126/RV1109 do not support aes-192 */
    #if defined(CONFIG_ROCKCHIP_RV1126)
    if (algo == CRYPTO_AES && key_len == AES_KEYSIZE_192)
        return -EINVAL;
    #endif

    ret = hw_cipher_init(g_key_chn, key, NULL, key_len, nonce, nonce_len,
                        algo, mode, false);
    if (ret)
        return ret;

    return hw_cipher_crypt(in, out, len, aad, aad_len,
                          tag, AES_BLOCK_SIZE, mode);
}
```

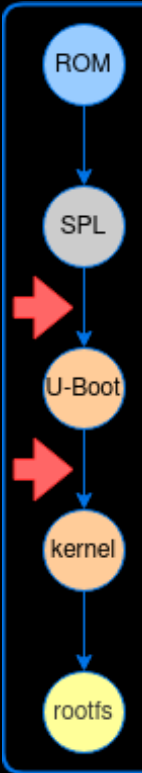
true = encrypt
false = decrypt

```
memcpy(nonce,src,FIT_ENC_KEY_SIZE);
memcpy(tag,src+FIT_ENC_KEY_SIZE,FIT_ENC_KEY_SIZE);
```

```
dev = crypto_get_device(CRYPTO_AES);
ctx.algo = CRYPTO_AES;
ctx.mode = RK_MODE_GCM;
ctx.key = key;
ctx.key_len = FIT_ENC_KEY_SIZE;
ctx.iv = &nonce[0];
ctx.iv_len = FIT_ENC_KEY_SIZE;
len-=2*FIT_ENC_KEY_SIZE;
```

```
int ret = crypto_ae(dev,&ctx,src+2*FIT_ENC_KEY_SIZE,len, aad, 0x10, src, dig);
uint64_t * a = (uint64_t *) tag;
uint64_t * b = (uint64_t *) dig;
if (ret || ((a[0]^b[0])|(a[1]^b[1]))) {
    memset(src,0,size+2*ENC_KEY_SIZE);
    memset(dst,0,size);
    return -2;
}
*src_len=len;
return 0;
```

Проверяем что тэг
совпал

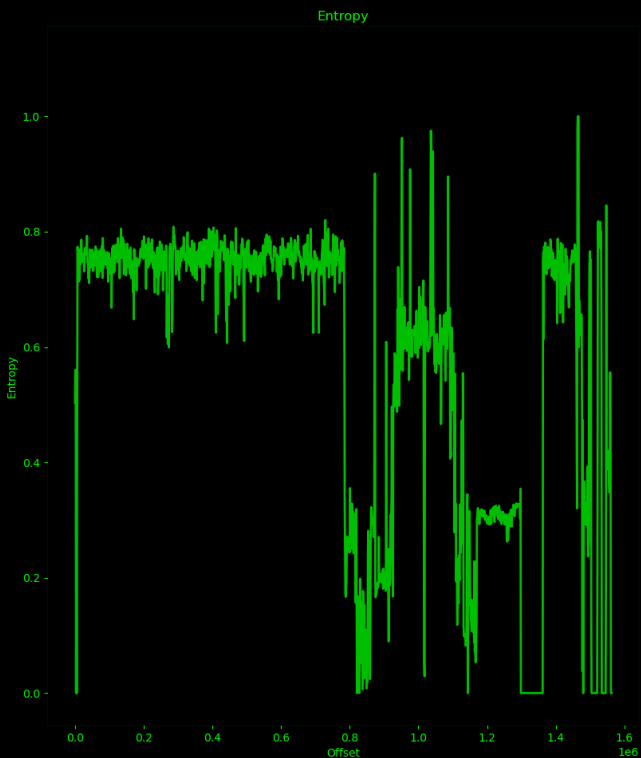


Encrypt more

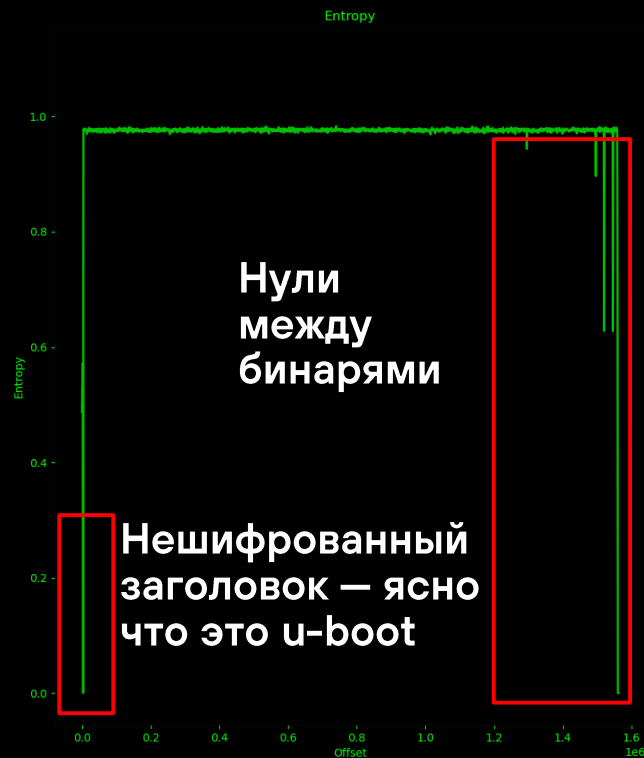
шифруем всё



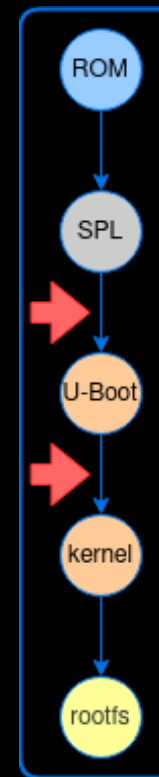
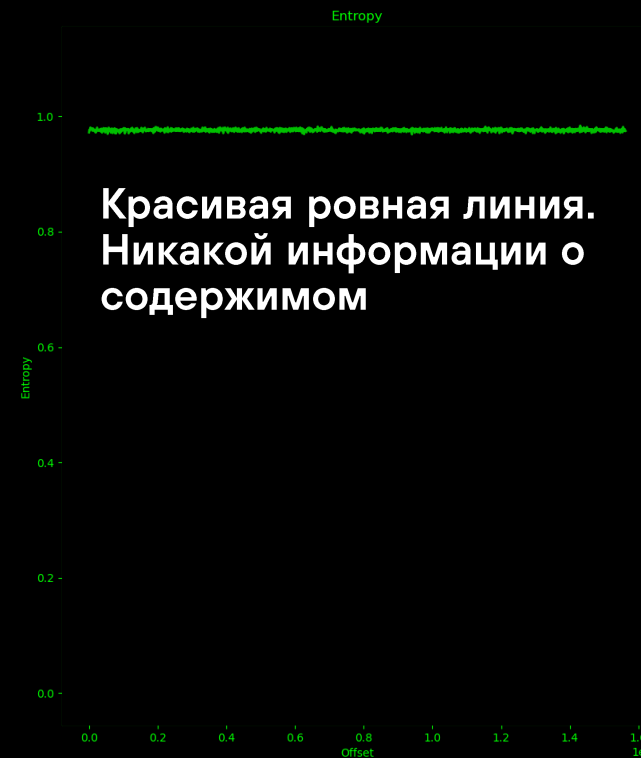
No encryption



Encrypted binaries



Full image encryption



Encrypt more

шифруем всё



arch/arm/mach-rockchip/fit.c

```
int decrypt(void* src, void* dst, size_t size, void* aad, void* tag_out){
    uint8_t* key;
    const void *blob = gd_fdt_blob();
    int key_node=fdt_subnode_offset(blob, 0, "encryption");
    key =(uint8_t *) fdt_getprop(blob, key_node, "key", NULL);

    uint8_t nonce[ENC_KEY_SIZE] = {0};
    uint8_t tag[ENC_KEY_SIZE]={0};
    uint8_t dig[ENC_KEY_SIZE]={0};
    struct udevice *dev;
    cipher_context ctx;

    memcpy(nonce,src,ENC_KEY_SIZE);
    memcpy(tag,src+ENC_KEY_SIZE,ENC_KEY_SIZE);

    dev = crypto_get_device(CRYPTO_AES);
    ctx.algo = CRYPTO_AES;
    ctx.mode = RK_MODE_GCM;
    ctx.key = key;
    ctx.key_len = ENC_KEY_SIZE;
    ctx.iv = &nonce[0];
    ctx.iv_len = ENC_KEY_SIZE;

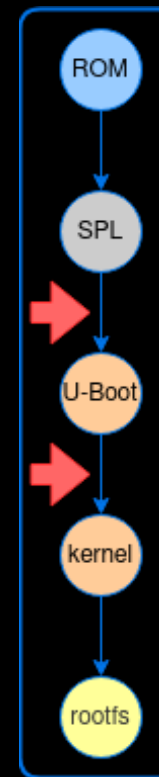
    int ret = crypto_ae(dev,&ctx,src+2*ENC_KEY_SIZE,size, aad, ENC_KEY_SIZE, dst, dig);
    uint64_t * a = (uint64_t *) tag;
    uint64_t * b = (uint64_t *) dig;
    if (ret || ((a[0]^b[0])|(a[1]^b[1]))) {
        memset(src,0,size+2*ENC_KEY_SIZE);
        memset(dst,0,size);
        return -2;
    }
    memset(dst+size,0,2*ENC_KEY_SIZE);
    if (tag_out)
        memcpy(tag_out,dig,ENC_KEY_SIZE);
    return 0;
}
```

arch/arm/mach-rockchip/fit.c : load bootables

```
if (blk_dread(dev_desc, part.start, blk_num, fit) != blk_num) {
    FIT_I("Failed to load bootable images\n");
    return NULL;
}
size_t head_size=sizeof(struct fdt_header); //0x28
uint8_t aad[ENC_KEY_SIZE]={0};
if (decrypt(fit,fit,head_size,aad,aad))
    return NULL;
size_t head2_size=fdt_totalsize(fit); //0x800
if (decrypt(fit+head_size+2*ENC_KEY_SIZE,fit+head_size,
    head2_size-head_size,aad,aad))
    return NULL;
if (decrypt(fit+head2_size+4*ENC_KEY_SIZE,fit+head2_size,
    *size-head2_size,aad,aad))
    return NULL;
return fit;
```

ЧТЕНИЕ
ВНЕШНЕЙ
ПАМЯТИ

"Зацепление" частей — мелочь, но красиво



LUKS

шифруем диск без пароля



Распаковываем initramfs

Добавляем root в crypttab

Добавляем ключ

Упаковываем initramfs

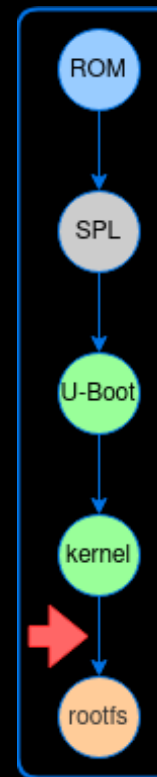
```
mkdir build/rd
cd build/rd
cpio -i -I ../kernel/rd
mkdir cryptroot
echo "root UUID=${1} /cryptroot/keyfiles/root.key initramfs,luks" > cryptroot/crypttab
mkdir cryptroot/keyfiles
cp ../../keys/luks/root.key cryptroot/keyfiles/
find . | cpio -o -H newc > ../kernel/ramdisk0
```

Создаём luks раздел

Записываем rootfs

Исправляем /boot

```
mount boot.img /mnt/p1
cryptsetup --key-file=keys/luks/root.key luksFormat /dev/mmcblk0p2
cryptsetup --key-file=keys/luks/root.key open /dev/mmcblk0p2 cryptosd
dd if=root.img of=/dev/mapper/cryptosd bs=64M status=progress
resize2fs /dev/mapper/cryptosd
mount /dev/mapper/cryptosd /mnt/p2
cp -r /mnt/p1/* /mnt/p2/boot/
vim /mnt/p2/etc/fstab # delete /boot entry
umount /mnt/p1
umount /mnt/p2
cryptsetup close cryptosd
```



SecureMaskROM

а вот и загрузчик



```
__int64 __fastcall load_binary__(header_struct *header, char *data, header_2_struct *fw_desc)
{
    int size; // w21
    unsigned int v6; // w0
    unsigned int v7; // w22
    _DWORD v9[4]; // [xsp+0h] [xbp-60h] BYREF
    char v10[32]; // [xsp+10h] [xbp-50h] BYREF
```

```
size = (LOBYTE(fw_desc->data_size) | (HIBYTE(fw_desc->data_size) << 8)) << 9;
```

```
if ( (header->boot_flag & 0x1000) != 0 )
```

Флаг определяет в какую ветку мы попадём

```
{
    v9[0] = (header->field_18[0] | (header->field_18[1] << 8))
            | ((header->field_18[2] | (header->field_18[3] << 8)) << 0x10);
    v9[1] = (header->field_18[4] | (header->field_18[5] << 8))
            | ((header->field_18[6] | (header->field_18[7] << 8)) << 0x10);
    v9[2] = 0;
    v9[3] = bswap32(
        (fw_desc->field_c[0] | (fw_desc->field_c[1] << 8))
        | ((fw_desc->field_c[2] | (fw_desc->field_c[3] << 8)) << 0x10));
    if ( !unk_FF00002C )
    {
        v7 = sub_FFFF0F98(0x20, 4u, 0);
        if ( v7 )
            return v7;
        unk_FF00002C = 1;
    }
}
```

Какая-то подготовка

```
v6 = sub_FFFF5C5C(0, v9, data, size, v10);
```

Что-то с hw_crypto

```
else
```

```
{
    v6 = sub_FFFF60B0(data, size, v10);
```

Что-то с hw_crypto, но поменьше

```
v7 = v6;
```

```
if ( !v6 )
```

```
{
```

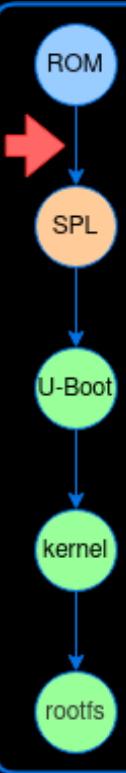
```
    v7 = sec_memcmp(fw_desc->field_18, v10, 0x20);
```

```
    sub_FFFF37B0(*word_FF000270 + 1, v7);
```

```
}
```

```
return v7;
```

```
}
```



SecureMaskROM

nonce

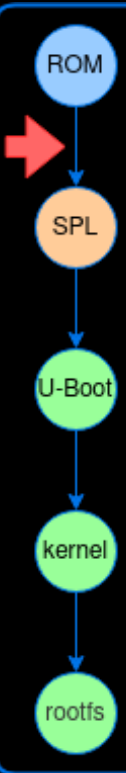
```
__int64 __fastcall load_binary__(header_struct *header, char *data, header_2_struct *fw_desc)
{
    int size; // w21
    unsigned int v6; // w0
    unsigned int v1; // w22
    _DWORD iv[4]; // [xsp+0h] [xbp-60h] BYREF
    char hash[32]; // [xsp+10h] [xbp-50h] BYREF

    size = (LOBYTE(fw_desc->data_size) | (HIBYTE(fw_desc->data_size) << 8)) << 9;
    if ( (header->boot_flag & 0x1000) != 0 )
    {
        iv[0] = (header->nonce[0] | (header->nonce[1] << 8)) | ((header->nonce[2] | (header->nonce[3] << 8)) << 0x10);
        iv[1] = (header->nonce[4] | (header->nonce[5] << 8)) | ((header->nonce[6] | (header->nonce[7] << 8)) << 0x10);
        iv[2] = 0;
        iv[3] = bswap32(
            (fw_desc->counter[0] | (fw_desc->counter[1] << 8))
            | ((fw_desc->counter[2] | (fw_desc->counter[3] << 8)) << 0x10));
        if ( !unk_FF00002C )
        {
            v1 = sub_FFFF0F98(0x20, 4u, 0);
            if ( v1 )
                return v1;
            unk_FF00002C = 1;
        }
        v6 = decrypt_and_hash(0, iv, data, size, hash);
    }
    else
    {
        v6 = ::hash(data, size, hash);
    }
    v1 = v6;
    if ( !v6 )
    {
        v1 = sec_memcmp(fw_desc->hash, hash, 0x20);
        sub_FFFF37B0(*word_FF000270 + 1, v1);
    }
    return v1;
}
```

nonce

Counter - сумма размеров предыдущих частей

Что-то странное



SecureMaskROM

keyladder



Что-то странное:
функция загрузки ключа

```
if ( !is_key_loaded )
{
    key = load_key(0x20, 4u, 0);
    if ( key )
        return key;
    is_key_loaded = 1;
}
```

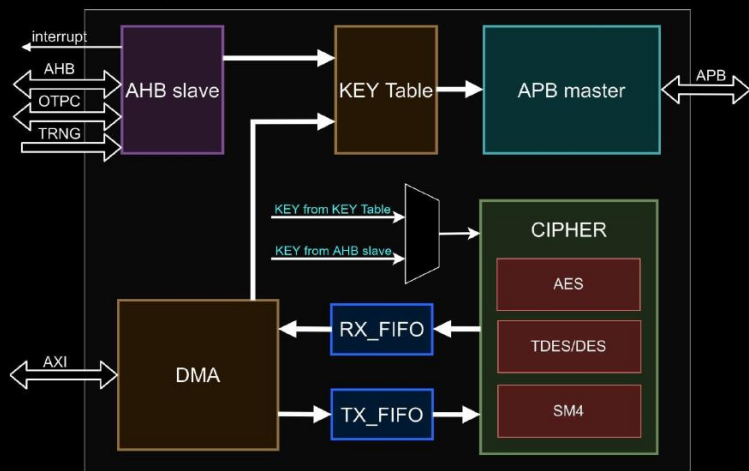
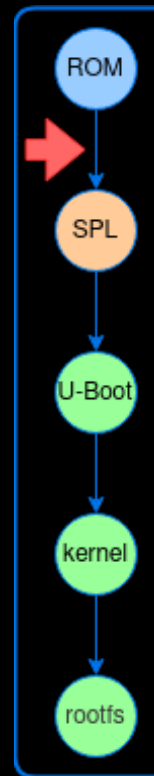


Fig. 33-1 KEYLAD Architecture

```
__int64 __fastcall load_key(int otp, unsigned int size, unsigned int a3)
{
    unsigned int v6; // w23
    unsigned int v7; // w22

    unk_FE380620 = 0;
    unk_FE380624 = 1;
    if ( size )
    {
        while ( 1 )
        {
            v6 = size >= 0x10 ? 0x10 : size;
            v7 = SECotpRead(0, otp, v6);
            if ( (v7 & 0x80000000) != 0 )
                break;
            size -= v6;
            otp += v6;
            if ( !size )
                goto LABEL_9;
        }
    }
    else
    {
        v7 = 0xFFFFFFFF;
    }
    LABEL_9:
    sub_FFFF4924(a3);
    return v7;
}
```

Чтение фьюзов в nullptr
Если не знать про
keyladder, выглядит
немного странно



Rock eBOOT

упаковщик



Шифруем бинари.

Формируем блок публичного ключа в Rockchip-овом формате

Формируем основной заголовок. В него входит nonce, флаги, включая флаг eBoot-a и указание того, что в прошивке 2 бинаря

Формируем заголовки бинарей. Добавляем хэши бинарей, и counter для расшифровки AESa

Подписываем хэш заголовка

packer.py

```
def pack(rsakey, aeskey, ddr, spl):
    nonce=get_random_bytes(8)
    aeskey1=open(aeskey, 'rb').read()
    aeskey=aeskey1[0:4][::-1] + aeskey1[4:8][::-1] + aeskey1[8:0xc][::-1] + aeskey1[0xc:0x10][::-1]
    ddr=open(dds, 'rb').read()
    if len(dds)%0x800:
        ddr+=b'\x00'*(0x800-len(dds)%0x800)
    a=AES.new(aeskey, AES.MODE_CTR, nonce=nonce)
    ddr=a.encrypt(dds)

    fw=open(spl, 'rb').read()
    if len(fw)%0x800:
        fw+=b'\x00'*(0x800-len(fw)%0x800)
    fw=a.encrypt(fw)

    key=RSA.importKey(open(rsakey, 'rb').read())
    keyblock=(key.n).to_bytes(0x100)[::-1]
    keyblock+=b'\x00'*0x100
    keyblock+=(key.e).to_bytes(0x10)[::-1]
    keyblock+=(pow(2, 2048+132)//key.n).to_bytes(0x100)[::-1]
    keyblock+=b'\x00'*0xf0

    flags=0x1011
    header=pad(struct.pack("<4sIHIII8s", magic, 0, size, 2, flags, 0, 0, nonce), 0x78)

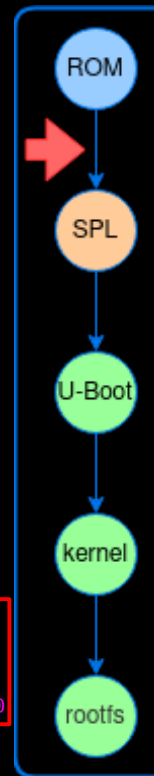
    ddr_hash=SHA256.new(dds).digest()
    header+=struct.pack("<HHIII", 4, len(dds)//0x200, 0xffff_ffff, 0, 0)+b'\x00'*8+ddr_hash+b'\x00'*0x20

    fw_hash=SHA256.new(fw).digest()
    header+=struct.pack("<HHIII", 4, len(dds)//0x200, len(fw)//0x200, 0xffff_ffff, 0x02, len(dds)//0x10)+b'\x00'*8+fw_hash+b'\x00'*0x20

    header=pad(header, 0x200)
    header+=keyblock

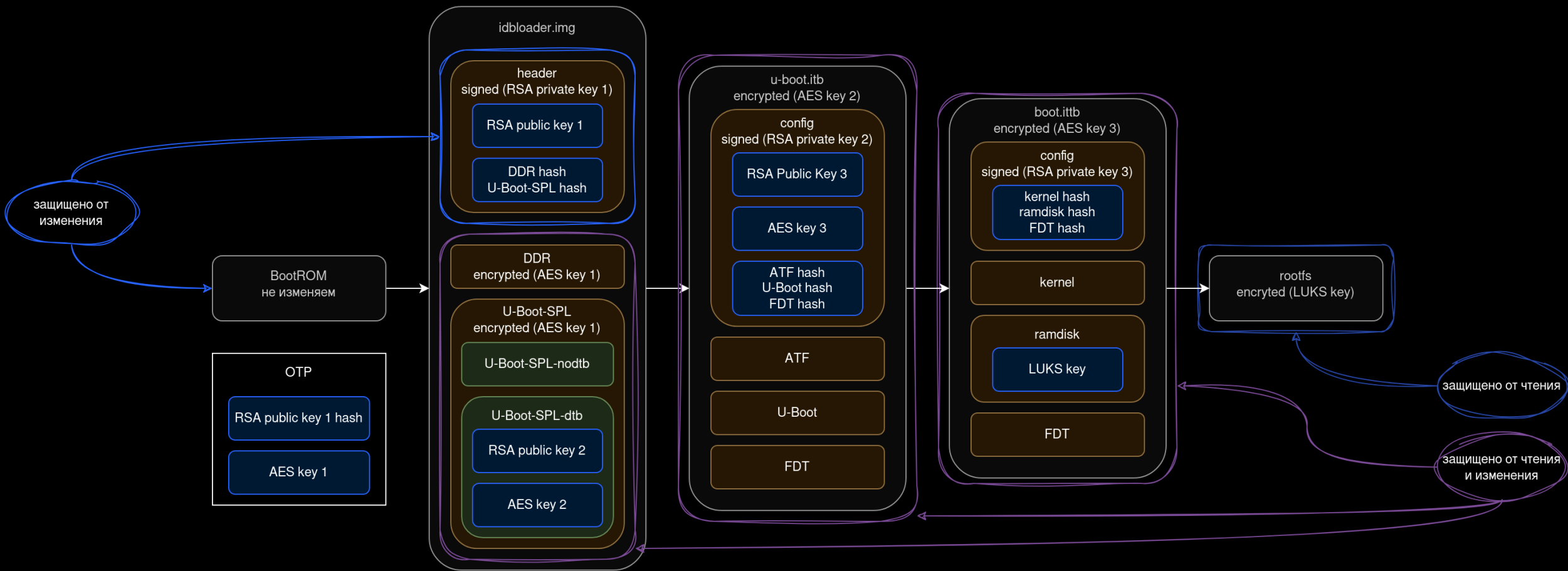
    a=PKCS1_PSS.new(key)
    head_hash=SHA256.new(header)
    sign=a.sign(head_hash)

    return header+sign[::-1]+b'\x00'*0x100+dds+fw
```



Encrypted BOOT

как выглядит цепочка





RK35 Family

портирование на другие SOC

RK3566

Easy port

```
int rsa_burn_key_hash(struct image_sign_info *info)
{
    char *rsa_key;
    void *n, *e, *c;
    uint32_t key_len;
    struct udevice *dev;
    struct key_prop prop;
    char name[100] = {0};
    u16 secure_flags = 0;
    const void *blob = info->fdt_blob;
    uint8_t digest[FIT_MAX_HASH_LEN];
    uint8_t digest_read[FIT_MAX_HASH_LEN];
    int sig_node, node, digest_len, i;
    int ret = 0, written_size = 0;

    /* Check burn-key-hash flag in itb first */
    sig_node = fdt_subnode_offset(blob, 0, FIT_SIG_NODENAME);
    if (sig_node < 0) {
        debug("%s: No signature node found\n", __func__);
        return -ENOENT;
    }

    snprintf(name, sizeof(name), "key-%s", info->keyname);
    node = fdt_subnode_offset(blob, sig_node, name);

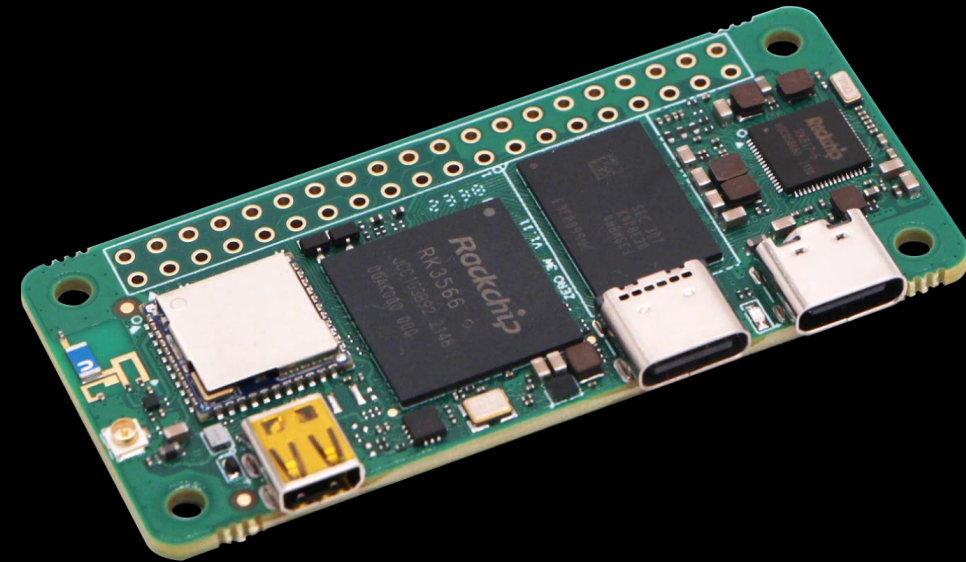
    if (rsa_get_key_prop(&prop, info, node))
        return -1;

    if (!(prop.burn_key))
        return 0;

    /* Handle burn_key_hash process from now on */
    dev = misc_otp_get_device(OTP_S);

    secure_flags = 0xff;
    ret = misc_otp_write(dev, OTP_SECURE_BOOT_ENABLE_ADDR,
                        &secure_flags, OTP_SECURE_BOOT_ENABLE_SIZE);
    if (ret)
        goto error;

    if (written_size)
        printf("RSA: Repair RSA key hash successfully.\n");
    else
        printf("RSA: Write RSA key hash successfully.\n");
}
```



Radxa Zero 3W



RK3566

Easy port

```
int length;
uint8_t* key_enc;
uint8_t key_enc_otp[16];
const void *key_blob = gd_fdt_blob();
int key_node=fdt_subnode_offset(key_blob, 0, "encryption");
key_enc =(uint8_t *) fdt_getprop(key_blob, key_node, "key", &length);

printf("enc key len %d\n",length);
ret = misc_otp_read(dev, 0xa*4 , key_enc_otp , 16);
if (ret)
    goto error;
if (memcmp(key_enc,key_enc_otp,16)==0){
    printf("enc key already written\n");
}else{
    for (int i=0;i<16;++i){
        if (key_enc_otp[i]!=0){
            printf("bad data in enc key otp\n");
            ret=-1;
            goto error;
        }
    }
    ret = misc_otp_write(dev, 0xa*4 , key_enc , 16);
    ret = misc_otp_read(dev, 0xa*4 , key_enc_otp , 16);
    dump("enc_key otp",key_enc_otp,length);
    if (memcmp(key_enc,key_enc_otp,16)!=0){
        printf("written enc key mismatch\n");
        return -1;
    }
    printf("write enc key OK\n");
}
```



Включение конфигов SecureBoot увеличивает размер SPL.

При сборке есть проверка что SPL не вылезает за границы своей области и не будет затёрт при чтении U-boot-a.

Но эта проверка бесполезна так как не учитывает FDT лежащий за SPL



RK35xx Secure Boot Bug

TOC-TOU in load_Storage

```
err = (src_descr->read_page)(v2, &fw_header_1, &v12); // read header to FF00_0040
if ( (err & 0x80000000) != 0 )
    break;
if ( !err )
{
    if ( ((aRknsrkss[0] | (aRknsrkss[1] << 8)) | ((aRknsrkss[2] | (aRknsrkss[3] << 8)) << 0x10)) == fw_header_1.magic
        || ((aRknsrkss[4] | (aRknsrkss[5] << 8)) | ((aRknsrkss[6] | (aRknsrkss[7] << 8)) << 0x10)) == fw_header_1.magic )
    {
        BOOT_CTX.boot_src_code = src_descr->code;
        dword_FF000014 = v12;
        p_CTX = src_descr;
        v4 = use_RKPK;
        v5 = use_RKPK != 0;
        if ( data_buff + (v5 << 0xB) + 0x800 <= 0xFF100000 )
            (src_descr->read)(v12, data_buff, (v5 << 0xB) + 0x800); // read_header to 0xff00_1000
        err = verify(data_buff); // check signature of header at 0xff00_1000
        if ( !err )
        {
            v6 = v4 ? 3 : 2;
            err = sub_FFFF36A8(0, v6);
            if ( !err )
            {
                func_20 = src_descr->func_20;
                if ( func_20 )
                    func_20(&fw_header_1.reserved2[0x10]);
                if ( data_buff + (fw_header_1.field_78[0].data_size << 9) <= 0xFF100000 )
                {
                    err = (p_CTX->read)(
                        dword_FF000014 + fw_header_1.field_78[0].off,
                        data_buff,
                        fw_header_1.field_78[0].data_size << 9);
                    if ( !err )
                    {
                        err = load_binary__(&fw_header_1, data_buff, fw_header_1.field_78); // use header at 0xff00_0040
                        if ( !err )
                        {
                            v0 = v4 ? 4 : 3.
                        }
                    }
                }
            }
        }
    }
}
```

header read 1

Уязвимые чипы

RK3588S: 350B 20210512 V100

RK3566: 350A 20210322 V300

header read 2
signature check

Usage of header from first read

RK3566 boot sources

Снова шьём фьюзы

```
if ( !get_Boot_mode() )
{
    sub_FFFF53A0();
    MEMORY[0xFDC60840] = 0x3F0003;
    MEMORY[0xFDC60844] = 0x3F0003;
    MEMORY[0xFDC60848] = 0x3F0003;
    MEMORY[0xFDC6084C] = 0x3F0003;
    MEMORY[0xFDC60850] = 0x3F0003;
    MEMORY[0xFDC60854] = 0x3F0003;
    MEMORY[0xFDC60228] = 0x3F000300;
    MEMORY[0xFDC60244] = 0x3F0003;
    MEMORY[0xFDC60250] = 0x3F0003;
    MEMORY[0xFDC60230] = 0xFFFF0F0F;
    MEMORY[0xFDC60234] = 0xFFFF0F0F;
    MEMORY[0xFDC60238] = 0xFFFF0F0F;
    MEMORY[0xFDC6023C] = 0xFFFF0F0F;
    boot_Storage();
    boot_USB();
}
while ( 1 )
;
```

```
result = OTP_Read(&v4, 0x44, 1u, 1u);
if ( !result )
{
    v4 &= 0xFu;
    if ( v4 && v4 < 6 || v4 == 0xA )
    {
        result = 0;
        Boot_mode = v4;
    }
    else
    {
        return 0;
    }
}
```

```
_int64 boot_Storage()
{
    int v0; // w9
    load_ctx *boot_src; // x20
    unsigned int Storage; // w19
    __int64 i; // x21
    void (*cleanup)(void); // x8

    if ( !Boot_mode )
    {
        i = 0;
        do
        {
            Storage = (boot_sources[i].check)();
            if ( Storage )
            {
                cleanup = boot_sources[i].cleanup;
                if ( cleanup )
                    cleanup();
            }
            else
            {
                Storage = load_Storage(&boot_sources[i]);
            }
            ++i;
        }
        while ( (i * 0x30) != 0xF0 );
        return Storage;
    }
    v0 = 0;
    for ( boot_src = boot_sources; boot_src->code != Boot_mode; ++boot_src )
    {
        if ( ++v0 >= 5 )
            return 0xFFFFFFFF;
    }
    Storage = (boot_src->check)();
    if ( Storage )
        return Storage;
    return load_Storage(boot_src);
}
```

RK3566: 350A 20210322 V300

Можно обойти баг

1...5

RK3576

Всё также, но по-другому



SB включается как на RK3566

Новый бинарь в idbloader.img: boost

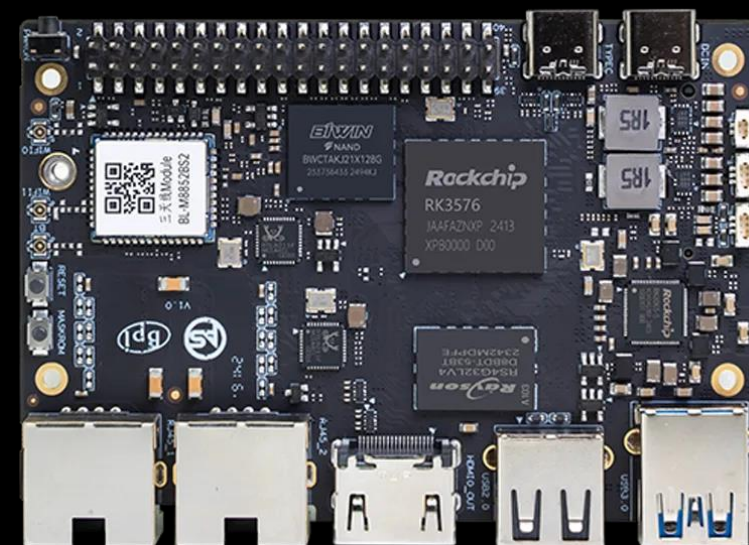
Другая схема адресного пространства

TRM не утёк

1 MaskROM в 0x0

EncryptedBoot есть

Китайская крипто при желании



Banana Pi M5-PRO

RK3576

Всё также, но по-другому

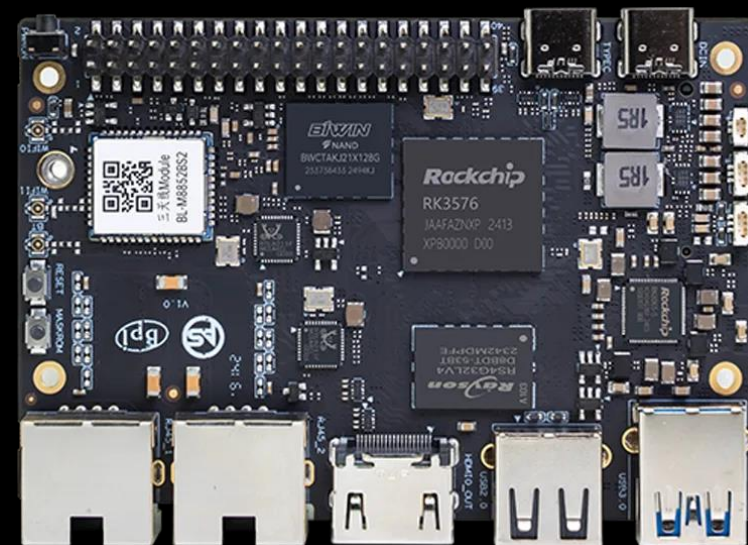


```
sub_4A68();
BOOT_CTX.secureboot = 0x3CA5;
dword_3FF80080 = 0xFFFFFFFF;
LOBYTE(Boot_addr_OK4) = 0;
LOBYTE(Boot_addr_OK3ff81) = 0;
*&BOOT_CTX.some_flags = 0;
dword_3FF80020 = 0;
BOOT_CTX.boot_src_code = 0;
nullsub_3();
sub_8264();
if ( check_secureboot_mode() )
{
    while ( 1 )
        sub_152C();
}
init_patch_rom();
if ( dword_26026200[0] == 0xEF08A53C )
{
    boot_src_seq = get_boot_src_seq(1, &boot_src_count);
}
else
{
    if ( dword_26026200[0] >> 4 == 0xEF085A3 )
        boot_src_seq = get_boot_src_seq(dword_26026200[0] & 0xF, &boot_src_count);
    else
        boot_src_seq = get_boot_src_seq_from_otp(&boot_src_count);
    v2 = boot_src_count;
    if ( boot_src_count )
    {
        boot_src_code = *boot_src_seq;
        if ( (boot_src_code + 0x7F) >= 3u )
        {
            v5 = 1;
            do
            {
                try_boot(boot_src_code);
                v2 = boot_src_count;
                if ( v5 >= boot_src_count )
                    goto LABEL_15;
                boot_src_code = boot_src_seq[v5++];
            }
            while ( (boot_src_code + 0x7F) > 2u );
            v4 = v5 - 1;
        }
        else
        {
            v4 = 0;
        }
        boot_src_seq += v4;
        boot_src_count = v2 - v4;
    }
}
LABEL_15:
sub_10F4();
boot_USB(boot_src_seq, boot_src_count);
}
```

```
// === READ 1 ===
err_code = (al->read_page_)(page_num, p_buffer_read_1, &paddr);
// === READ 1 ===
if ( err_code )
{
    set_boot_ctx_flag(4);
}
else
{
    magic = (*p_buffer_read_1 | (p_buffer_read_1[1] << 8)
        | ((p_buffer_read_1[2] | (p_buffer_read_1[3] << 8)) << 0x10);
    if ( magic == 'SNKR' || magic == 'SSKR' )
    {
        BOOT_CTX.boot_src_code = boot_src_code;
        BOOT_CTX.paddr = paddr;
        memcpy(&fw_header_struct, p_buffer_read_1, 0x200u);
        clear_boot_ctx_flag(4);
    }

    if ( p_BOOT_CTX->secureboot )
    {
        err_code = 0xFFFFFFFF;
        if ( (fw_header_struct.nimages & 0xFu) <= 4 )
        {
            nimages = fw_header_struct.nimages & 0xF;
            load_addr = p_buffer_read_2;

            use_rkpk = use_RKPK[0];
            block_size = al->block_size__;
            v13 = use_RKPK[0] != 0;
            if ( use_RKPK[0] )
                header_size = 0x1000;
            else
                header_size = 0x800;
            if ( header_size > block_size )
            {
                read = read_func;
                if ( !read_func )
                    read = al->read_;
                if ( header_size >= block_size )
                    header_read_size = header_size;
                else
                    header_read_size = al->block_size__;
                // === READ 2 ===
                read(BOOT_CTX.paddr, p_buffer_read_2, header_read_size);
                // === READ 2 ===
            }
            v17 = verify(p_buffer_read_2);
            if ( v17 )
            {
                err_code = v17;
                v18 = 0x80;
                goto Error_1;
            }
            memcpy(&fw_header_struct, buffer, 0x200u);
        }
    }
}
```



RK3576: 350E 20231110 V100

Баг исправлен

Rock eBOOT release

POC edition 😊



Теперь есть всё что нужно

Я, конечно, тестировал, но баги могли остаться

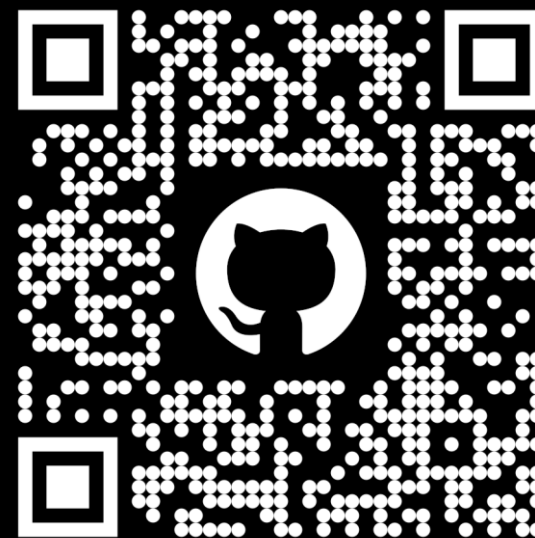
Зашивка ключей через специальный U-boot-SPL

Пока только RK3588, но порт на RK3566/RK3576 лёгкий

Автор не несёт никакой ответственности за окирпиченные платы

Думайте что делаете! 7 раз проверь, 1 прошей

Пакер под RK3576 постараюсь выложить



Conclusion



Use eBoot to protect your firmware

No docs? Reverse BootROM

Even small code can be vulnerable

Opensource eBoot for RK35*

Use at your own risk

Q&A

“HR” Positive Labs

